Exposing the data to Payload (headless CMS)

The goal: let editors reference real geography metrics inside structured content fields, and let downstream content-generation / modeling read clean, typed values — without coupling the CMS to the ingestion internals.

The contract is the views, not the tables

Payload reads from `v_acs_latest` (and friends), never from `acs_observations` directly. This means we can re-partition, promote new JSONB fields, or change batch internals without touching any CMS config. The views are the stable API.

Two integration shapes

1. Read-only reference data (recommended start)

ACS metrics are reference data the CMS *reads* but doesn't *own*. The cleanest wiring is a thin read API (or a Payload custom endpoint / database adapter view) backed by `v_acs_latest`, surfaced as a **relationship-like select** in content fields:

- Editor picks a geography (county) in a content field.
- The field stores the stable `geoid`.
- Render/generation time, the platform joins `geoid → v_acs_latest` for `total_population`, `median_household_income`, etc.

Storing the `geoid` (not a snapshot of the values) means content always reflects the latest loaded vintage.

2. Mirrored Payload collection

If editors need to browse/search geographies inside Payload's admin, define a read-mostly collection whose fields map 1:1 to the promoted columns:

```
// payload.config.ts — illustrative
const Counties: CollectionConfig = {
  slug: 'counties',
  admin: { useAsTitle: 'name', defaultColumns: ['name', 'totalPopulation'] },
  access: { create: () => false, update: () => false, delete: () => false },
  fields: [
    { name: 'geoid', type: 'text', unique: true, index: true, required: true },
    { name: 'name', type: 'text' },
    { name: 'totalPopulation', type: 'number' },
    { name: 'medianHouseholdIncome', type: 'number' },
    { name: 'medianHomeValue', type: 'number' },
    // Long-tail ACS fields stay in Postgres JSONB; expose on demand as a
    // read-only JSON field if a workflow genuinely needs them in-admin.
    { name: 'raw', type: 'json', admin: { readOnly: true } },
  ],
}
```

The pipeline keeps this collection in sync via the same upsert key (`geoid`), either by pointing Payload's DB adapter at the view or by a small sync step that writes the promoted columns into the collection after each load.

Field-mapping table

Postgres (view `v_acs_latest`)	Type	Payload field	Notes
`geoid`	text	`geoid` (text, unique, index)	join key / content reference
`name`	text	`name` (text)	`useAsTitle`
`total_population`	bigint	`totalPopulation` (number)	filter/sort
`median_household_income`	int	`medianHouseholdIncome` (number)	filter/sort
`median_home_value`	int	`medianHomeValue` (number)	
`median_gross_rent`	int	`medianGrossRent` (number)	
`unemployed_count`	int	`unemployedCount` (number)	
`bachelors_count`	int	`bachelorsCount` (number)	
`raw`	jsonb	`raw` (json, readOnly)	optional; long-tail fields

Guidance

- **Promote before you expose.** A field that needs to be filterable/sortable in Payload should be a typed column first (see DESIGN.md) — don't make the CMS reach into JSONB for hot fields.
- **Reference by `geoid`, not by snapshot**, so content tracks the latest load.
- **Keep the CMS read-only** over ingested data; the pipeline is the source of truth and the only writer.